

MARAN PMS — Módulo Contable/Administrativo

PROMPT A: Base de Datos — Proveedores, Facturas de Compra, Plan de Cuentas

CONTEXTO

Estás trabajando en el sistema PMS del Hotel Maran Suites & Towers (Paraná, Entre Ríos). Stack: PostgreSQL + Drizzle ORM + React + TypeScript. CUIT del hotel: 33-68110008-9 | Razón social: MARAN SA

Este prompt crea la estructura de base de datos completa para el módulo de **Compras/Administración**. No conecta con ARCA todavía — los documentos se generan internamente para validación.

PARTE 1 — TABLA DE PROVEEDORES

Crear tabla `suppliers` en `shared/schema.ts`:

```
export const suppliers = pgTable("suppliers", {
  id: serial("id").primaryKey(),
  razonSocial: text("razon_social").notNull(),
  cuit: text("cuit").notNull().unique(), // formato: XX-XXXXXXX-X
  domicilio: text("domicilio"),
  localidad: text("localidad"),
  provincia: text("provincia").default("Entre Rios"),
  cp: text("cp"),
  condicionIva: text("condicion_iva").notNull(), // "R.Inscrip." | "Monotributo"
  // Retenciones
  alicuotaIibb: numeric("alicuota_iibb", { precision: 6, scale: 4 }).default("0"),
  alicuotaGanancias: numeric("alicuota_ganancias", { precision: 6, scale: 4 }).default("0"),
  alicuotaIva: numeric("alicuota_iva", { precision: 6, scale: 4 }).default("0"),
  // Pago
  cbu: text("cbu"),
  banco: text("banco"),
  // Control
```

```

    activo: boolean("activo").default(true),
    createdAt: timestamp("created_at").defaultNow(),
    updatedAt: timestamp("updated_at").defaultNow(),
  });

```

PARTE 2 — TABLA DE CUENTAS CONTABLES

Crear tabla `accounting_accounts` **para el plan de cuentas:**

```

export const accountingAccounts = pgTable("accounting_accounts", {
  id: serial("id").primaryKey(),
  codigo: text("codigo").notNull().unique(), // ej: "1.1.4.07.01", "4.2.1.08.27"
  nombre: text("nombre").notNull(),           // ej: "IVA 21%", "Costo Insumo y c
  tipo: text("tipo").notNull(),               // "activo" | "pasivo" | "patrimoni
  nivel: integer("nivel").default(1),        // profundidad del plan: 1=mayor, 2
  activo: boolean("activo").default(true),
});

```

Insertar el plan de cuentas inicial con los datos reales del hotel (extraídos del Mayor de Cuentas Junio 2025):

```

INSERT INTO accounting_accounts (codigo, nombre, tipo) VALUES
-- Activos (impuestos a favor / créditos fiscales)
('1.1.4.01.04.01', 'Ret. IVA', 'activo'),
('1.1.4.01.04.02', 'Percep IVA', 'activo'),
('1.1.4.01.05', 'Ret Impuestos a las ganancias', 'activo'),
('1.1.4.01.08.01', 'Ret. Ing Brutos', 'activo'),
('1.1.4.01.08.02', 'Percep Ing Brutos', 'activo'),
('1.1.4.01.10', 'Retenciones SUSS', 'activo'),
('1.1.4.01.15', 'Impuesto Ley 25413', 'activo'),
('1.1.4.07.01', 'IVA 21%', 'activo'),
('1.1.4.07.02', 'IVA 10,5%', 'activo'),
('1.1.4.07.03', 'IVA 27%', 'activo'),
-- Pasivos
('2.1.3.02.09', 'Impuestos Internos', 'pasivo'),
-- Egresos / Gastos
('4.2.1.08.05.02', 'Gastos Comerciales', 'egreso'),
('4.2.1.08.06.01', 'Librería', 'egreso'),
('4.2.1.08.06.02', 'Imprenta', 'egreso'),
('4.2.1.08.07.02', 'Lavandería Hotel', 'egreso'),
('4.2.1.08.09.01', 'Combustibles', 'egreso'),
('4.2.1.08.09.05', 'Gastos Generales', 'egreso'),
('4.2.1.08.10.06', 'Gastos del Personal', 'egreso'),

```

```

('4.2.1.08.11.01', 'Luz', 'egreso'),
('4.2.1.08.11.02', 'Gas', 'egreso'),
('4.2.1.08.11.03', 'Teléfono', 'egreso'),
('4.2.1.08.12', 'Publicidad Maran Towers', 'egreso'),
('4.2.1.08.13.01', 'Honorarios', 'egreso'),
('4.2.1.08.14', 'Fletes y Franqueos Maran Tower', 'egreso'),
('4.2.1.08.15.01', 'Municipalidad', 'egreso'),
('4.2.1.08.16', 'Mantenimiento Bs de Uso Maran', 'egreso'),
('4.2.1.08.17', 'Gastos Computación Maran Tower', 'egreso'),
('4.2.1.08.18', 'Gastos Bancarios Maran Towers', 'egreso'),
('4.2.1.08.22', 'Mantenimiento Edificio Maran T', 'egreso'),
('4.2.1.08.23.01', 'Housekeeping', 'egreso'),
('4.2.1.08.23.02', 'Housekeeping Cocina', 'egreso'),
('4.2.1.08.23.03', 'Housekeeping Áreas Públicas', 'egreso'),
('4.2.1.08.23.04', 'Housekeeping Lavandería', 'egreso'),
('4.2.1.08.24', 'Cablevideo', 'egreso'),
('4.2.1.08.27', 'Costo Insumo y comestibles', 'egreso'),
('4.2.1.08.28.01', 'Aves Restaurant', 'egreso'),
('4.2.1.08.28.03', 'Cerdo Restaurant', 'egreso'),
('4.2.1.08.28.05', 'Pescado Restaurant', 'egreso'),
('4.2.1.08.28.09', 'Vacuno Restaurant', 'egreso'),
('4.2.1.08.29.01', 'Fiambres Restaurant', 'egreso'),
('4.2.1.08.32.01', 'Frut V y H Restaurant', 'egreso'),
('4.2.1.08.33.01', 'Lácteos Restaurant', 'egreso'),
('4.2.1.08.35.01', 'Costo Bebidas Restaurant', 'egreso'),
('4.2.1.08.36', 'Otros Gastos Restaurant', 'egreso'),
('4.2.1.08.39', 'Otros Gastos Spa', 'egreso'),
-- Cuentas de pago / cancelación (para asientos)
('1.1.1.01', 'Caja', 'activo'),
('1.1.1.02', 'Banco Macro', 'activo'),
('1.1.1.03', 'Banco Santander', 'activo'),
('1.1.1.04', 'Banco Nuevo Bersa', 'activo'),
('1.1.1.05', 'Banco de la Nación', 'activo'),
-- Cuentas de proveedores (pasivo)
('2.1.1.01', 'Proveedores a Pagar', 'pasivo');

```

PARTE 3 — TABLA DE COMPROBANTES DE COMPRA

Crear tabla `purchase_invoices` — cubre facturas de proveedores (punto 1), resúmenes bancarios (punto 5) y liquidaciones de tarjeta (punto 6):

```

export const purchaseInvoices = pgTable("purchase_invoices", {
  id: serial("id").primaryKey(),

```

```

// Tipo de comprobante
tipoComprobante: text("tipo_comprobante").notNull(),
// Valores posibles:
// "FACT-A" | "FACT-B" | "FACT-C" | "NC-A" | "NC-B" | "NC-C" → facturas y nota
// "RESUMEN-BANCO" → resúmenes bancarios
// "LIQ-TARJETA" → liquidaciones de tarjeta

// Proveedor / entidad emisora
supplierId: integer("supplier_id").references(() => suppliers.id),
// Para resúmenes bancarios y tarjetas puede no haber un proveedor formal.
// En ese caso, guardar los datos directamente:
proveedorNombre: text("proveedor_nombre"), // redundante para facilitar búsqueda
proveedorCuit: text("proveedor_cuit"),

// Identificación del comprobante
puntoVenta: text("punto_venta"), // ej: "0023"
numeroComprobante: text("numero_comprobante").notNull(), // ej: "00016586"
numeroComprobanteExt: text("numero_comprobante_ext"), // número completo for
fechaEmision: date("fecha_emision").notNull(),
periodo: text("periodo"), // formato "MM/YYYY" ej: "01/2026"

// Condición de pago
condicionPago: text("condicion_pago").notNull().default("contado"),
// "contado" | "cuenta_corriente"

// Montos – todos en pesos, sin signo (las notas de crédito se identifican por
montoNeto: numeric("monto_net", { precision: 14, scale: 2 }).notNull().default
// Alícuotas de IVA
alicuotaIva: text("alicuota_iva").default("21"),
// "27" | "21" | "10.5" | "5" | "2.5" | "0" (exento) | "NG" (no gravado)
montoIva27: numeric("monto_iva27", { precision: 14, scale: 2 }).default("0"),
montoIva21: numeric("monto_iva21", { precision: 14, scale: 2 }).default("0"),
montoIva105: numeric("monto_iva105", { precision: 14, scale: 2 }).default("0"),
montoIva5: numeric("monto_iva5", { precision: 14, scale: 2 }).default("0"),
montoIva25: numeric("monto_iva25", { precision: 14, scale: 2 }).default("0"),
montoExento: numeric("monto_exento", { precision: 14, scale: 2 }).default("0"),
montoNoGravado: numeric("monto_no_gravado", { precision: 14, scale: 2 }).default

// Impuestos adicionales
impuestosInternos: numeric("impuestos_internos", { precision: 14, scale: 2 }).default
ley25413: numeric("ley_25413", { precision: 14, scale: 2 }).default("0"),
percepcionIibb: numeric("percepcion_iibb", { precision: 14, scale: 2 }).default
percepcionIva: numeric("percepcion_iva", { precision: 14, scale: 2 }).default("0")
percepcionGanancias: numeric("percepcion_ganancias", { precision: 14, scale: 2 }).default
retencionIibb: numeric("retencion_iibb", { precision: 14, scale: 2 }).default("0")
retencionGanancias: numeric("retencion_ganancias", { precision: 14, scale: 2 }).default
retencionIva: numeric("retencion_iva", { precision: 14, scale: 2 }).default("0")

```

```

    retencionSuss: numeric("retencion_suss", { precision: 14, scale: 2 }).default("
    retencionMunicipal: numeric("retencion_municipal", { precision: 14, scale: 2 })
    monotributoCompBC: numeric("monotributo_comp_bc", { precision: 14, scale: 2 }).

// Total del comprobante (calculado: neto + iva + adicionales)
montoTotal: numeric("monto_total", { precision: 14, scale: 2 }).notNull().defau

// Clasificación contable
cuentaContableId: integer("cuenta_contable_id").references(() => accountingAcco
centroCosto: text("centro_costo"),
// Valores sugeridos: "Hotel" | "Restaurant" | "Spa" | "Administración" | "Mant

// Estado
estado: text("estado").notNull().default("pendiente"),
// "pendiente" → cuenta corriente sin pagar
// "pagado"      → pagado (contado o CC cancelado con OP)
// "anulado"

// Asiento contable generado
asientoId: integer("asiento_id"), // FK a accounting_entries (se completa en P

// Metadata
observaciones: text("observaciones"),
createdAt: timestamp("created_at").defaultNow(),
updatedAt: timestamp("updated_at").defaultNow(),
});

```

PARTE 4 — TABLA DE ÓRDENES DE PAGO

Crear tabla `payment_orders` para las OP (punto 7):

```

export const paymentOrders = pgTable("payment_orders", {
  id: serial("id").primaryKey(),
  numero: text("numero").notNull().unique(), // formato: "000-00011471" (autoi

  supplierId: integer("supplier_id").notNull().references(() => suppliers.id),

  fecha: date("fecha").notNull(),

// Forma de pago
formaPago: text("forma_pago").notNull().default("transferencia"),
// "transferencia" | "efectivo" | "cheque"
depBancario: numeric("dep_bancario", { precision: 14, scale: 2 }).default("0"),
efectivo: numeric("efectivo", { precision: 14, scale: 2 }).default("0"),

```

```

cheques: numeric("cheques", { precision: 14, scale: 2 }).default("0"),

// Totales
totalFacturas: numeric("total_facturas", { precision: 14, scale: 2 }).notNull()
retencionIibb: numeric("retencion_iibb", { precision: 14, scale: 2 }).default("0")
retencionGanancias: numeric("retencion_ganancias", { precision: 14, scale: 2 })
retencionIva: numeric("retencion_iva", { precision: 14, scale: 2 }).default("0")
retencionProfLibs: numeric("retencion_prof_libs", { precision: 14, scale: 2 })
compensacion: numeric("compensacion", { precision: 14, scale: 2 }).default("0")
totalAbonado: numeric("total_abonado", { precision: 14, scale: 2 }).notNull(),

// Asiento contable
asientoId: integer("asiento_id"),

observaciones: text("observaciones"),
createdAt: timestamp("created_at").defaultNow(),
});

// Tabla intermedia: qué facturas cancela cada OP
export const paymentOrderItems = pgTable("payment_order_items", {
  id: serial("id").primaryKey(),
  paymentOrderId: integer("payment_order_id").notNull().references(() => paymentOrderItems, { onDelete: 'cascade' }),
  invoiceId: integer("invoice_id").notNull().references(() => purchaseInvoices.id, { onDelete: 'cascade' }),
  importeCancelado: numeric("importe_cancelado", { precision: 14, scale: 2 }).notNull(),
});

```

PARTE 5 — TABLA DE ASIENTOS CONTABLES

Crear tabla `accounting_entries` para el Mayor de Cuentas:

```

export const accountingEntries = pgTable("accounting_entries", {
  id: serial("id").primaryKey(),
  numeroMinuta: integer("numero_minuta").notNull(), // autoincremental por período
  fecha: date("fecha").notNull(),
  periodo: text("periodo").notNull(), // "MM/YYYY"
  concepto: text("concepto").notNull(),
  // ej: "Prov Fac/NDebito A/M" | "Prov.Liq.Tarjeta" | "Prov. Resumen Banco" | "P
  tipoOrigen: text("tipo_origen").notNull(),
  // "factura" | "nota_credito" | "orden_pago" | "retencion" | "manual"
  origenId: integer("origen_id"), // FK a purchase_invoices.id o payment_order_items.id
  origenTipo: text("origen_tipo"), // "purchase_invoice" | "payment_order_item"
  createdAt: timestamp("created_at").defaultNow(),
});

```

```
// Líneas del asiento (cada cuenta que debita o acredita)
export const accountingEntryLines = pgTable("accounting_entry_lines", {
  id: serial("id").primaryKey(),
  entryId: integer("entry_id").notNull().references(() => accountingEntries.id),
  accountId: integer("account_id").notNull().references(() => accountingAccounts.id),
  // Texto descriptivo del comprobante (para el mayor detallado)
  comprobanteTipo: text("comprobante_tipo"), // "PROV FACT -A-" | "PROV RESUMEN"
  comprobanteNumero: text("comprobante_numero"), // número del comprobante
  proveedorNombre: text("proveedor_nombre"),
  debe: numeric("debe", { precision: 14, scale: 2 }).default("0"),
  haber: numeric("haber", { precision: 14, scale: 2 }).default("0"),
});
```

PARTE 6 — TABLA DE CONSTANCIAS DE RETENCIÓN IIBB (SIRCAR)

Crear tabla `iibb_retentions` para guardar las retenciones generadas y exportar al SIRCAR:

```
export const iibbRetentions = pgTable("iibb_retentions", {
  id: serial("id").primaryKey(),
  nroConstancia: integer("nro_constancia").notNull(), // autoincremental global
  supplierId: integer("supplier_id").references(() => suppliers.id),
  cuitProveedor: text("cuit_proveedor").notNull(),
  fechaRetencion: date("fecha_retencion").notNull(),
  fechaComprobante: date("fecha_comprobante").notNull(),
  nroComprobante: integer("nro_comprobante").notNull(), // el nro interno de la f
  letraFactura: text("letra_factura"), // "A" | "B" | "C"
  importeBase: numeric("importe_base", { precision: 14, scale: 2 }).notNull(),
  alicuota: numeric("alicuota", { precision: 6, scale: 4 }).notNull(),
  importeRetenido: numeric("importe_retenido", { precision: 14, scale: 2 }).notNull(),
  anulacion: boolean("anulacion").default(false),
  convMultilateral: boolean("conv_multilateral").default(false),
  invoiceId: integer("invoice_id").references(() => purchaseInvoices.id),
  createdAt: timestamp("created_at").defaultNow(),
});
```

PARTE 7 — MIGRACIÓN

Crear y ejecutar la migración con Drizzle:

```
npx drizzle-kit generate
npx drizzle-kit push
```

Verificar en la consola que se crearon las tablas:

- suppliers
- accounting_accounts
- purchase_invoices
- payment_orders
- payment_order_items
- accounting_entries
- accounting_entry_lines
- iibb_retentions

PARTE 8 — SEED DE DATOS INICIALES (proveedores reales del hotel)

Insertar los proveedores principales extraídos del Listado de Retenciones IIBB real del hotel, para que el sistema tenga datos de prueba desde el primer día:

```
// En server/storage.ts o en un archivo seed separado
const proveedoresIniciales = [
  { razonSocial: "AGUA NUESTRA SA", cuit: "30-70786951-4", condicionIva: "R.Inscr" },
  { razonSocial: "BODEGAS CHANDON S.A.", cuit: "30-55371841-0", condicionIva: "R." },
  { razonSocial: "CAPUCHINO SRL", cuit: "33-71252875-9", condicionIva: "R.Inscrip." },
  { razonSocial: "Congelados Veracruz SRL", cuit: "30-71490852-5", condicionIva: "R." },
  { razonSocial: "DISTRIBUIDORA ANTARTIDA SA", cuit: "30-70848176-5", condicionIva: "R." },
  { razonSocial: "FRUTAS RAULITO SAS", cuit: "33-71846981-9", condicionIva: "R.Inscrip." },
  { razonSocial: "HUGO O ISAAC DISTRIB SRL", cuit: "30-70993544-1", condicionIva: "R." },
  { razonSocial: "L & L FUTURA SRL", cuit: "30-69338113-0", condicionIva: "R.Inscrip." },
  { razonSocial: "Lobo Gustavo Hernan", cuit: "20-27833483-0", condicionIva: "R.Inscrip." },
  { razonSocial: "Logistica San Miguel SRL", cuit: "30-70826260-5", condicionIva: "R." },
  { razonSocial: "MARTIN GABRIEL CIPRIANI", cuit: "20-22342199-8", condicionIva: "R." },
  { razonSocial: "NESTLE ARGENTINA S.A", cuit: "30-54676404-0", condicionIva: "R." },
  { razonSocial: "PALADINI SA", cuit: "30-50334872-8", condicionIva: "R.Inscrip." },
  { razonSocial: "PANIFICADORA RIO PARANA SA", cuit: "30-71573529-2", condicionIva: "R." },
  { razonSocial: "Parana Beef SRL", cuit: "30-71553991-4", condicionIva: "R.Inscrip." },
]
```



```

{ razonSocial: "SCHONFELD MAURICIO RUBEN", cuit: "20-25307793-0", condicionIva:
{ razonSocial: "Santana Javier Salvador", cuit: "20-21912741-4", condicionIva:
{ razonSocial: "Total Litoral SA", cuit: "30-71703401-1", condicionIva: "R.Insc
{ razonSocial: "Vazquez Maria Andrea", cuit: "27-22342883-0", condicionIva: "R.
{ razonSocial: "Virtu SRL", cuit: "30-71701621-8", condicionIva: "R.Inscrp.", a
{ razonSocial: "FONTANA ROLANDO RAUL", cuit: "20-12756016-2", condicionIva: "R.
{ razonSocial: "MATZKIN DARIO JAVIER", cuit: "20-16958502-5", condicionIva: "R.
{ razonSocial: "PAPELERIA EL CUYANO SRL", cuit: "30-67118492-7", condicionIva:
{ razonSocial: "MARCELO A. FERNANDEZ CUETO", cuit: "20-14310306-5", condicionIv
{ razonSocial: "Troceadero de Cerdos Don", cuit: "30-65553842-5", condicionIva:
{ razonSocial: "VYG SRL", cuit: "30-71738382-2", condicionIva: "R.Inscrp.", ali
{ razonSocial: "KORE SA", cuit: "30-71160470-3", condicionIva: "R.Inscrp.", ali
{ razonSocial: "ELECTRICIDAD PARANA S.A", cuit: "30-71440399-7", condicionIva:
{ razonSocial: "SELPLAST SA", cuit: "30-69050738-9", condicionIva: "R.Inscrp.",
{ razonSocial: "Gervasoni Pablo Cesar", cuit: "20-30786763-0", condicionIva: "M
{ razonSocial: "ANGEL H.BELLONI SRL", cuit: "30-64620565-0", condicionIva: "R.I
{ razonSocial: "FADEFIL SA", cuit: "30-71295020-6", condicionIva: "R.Inscrp.",
{ razonSocial: "ERTIC SRL", cuit: "30-71104918-1", condicionIva: "R.Inscrp.", a
{ razonSocial: "LA ALBORADA DEL SUD SA", cuit: "30-70828075-1", condicionIva: "
{ razonSocial: "Zurdo Nancy Viviana", cuit: "27-16996676-7", condicionIva: "R.I
{ razonSocial: "Cancio Eduardo", cuit: "20-16608544-7", condicionIva: "R.Inscrp
{ razonSocial: "LA RURAL VIÑEDOS Y BODEGAS", cuit: "30-52719611-2", condicionIv
{ razonSocial: "NOSTER SRL", cuit: "30-71151601-4", condicionIva: "R.Inscrp.",
{ razonSocial: "ALDO M.FERNANDEZ OSUNA", cuit: "20-24592976-6", condicionIva: "
{ razonSocial: "Ramirez Alberto Adrian", cuit: "20-22342219-6", condicionIva: "
];

```

Ejecutar el seed solo si la tabla `suppliers` está vacía:

```

const existing = await db.select().from(suppliers).limit(1);
if (existing.length === 0) {
  await db.insert(suppliers).values(proveedoresIniciales);
}

```

NOTAS IMPORTANTES

1. El campo `numeroComprobante` en `purchase_invoices` guarda solo el número (ej: "00016586"). El `puntoVenta` guarda el punto (ej: "0023"). El `numeroComprobanteExt` guarda la forma completa "0023-00016586" para búsquedas y exportaciones a ARCA.
2. Los asientos contables generados automáticamente al cargar una factura se describen en el **Prompt B**.

3. La lógica de cálculo de retenciones IIBB usa `alicuotaIibb` del proveedor x `montoNeto` de la factura. La encargada puede actualizar la alícuota por proveedor mensualmente desde el ABM de proveedores.
4. Las notas de crédito (`NC-A` , `NC-B` , `NC-C`) tienen `montoNeto` positivo pero se registran con signo negativo en los asientos, reduciendo el saldo del proveedor.
5. El `periodo` en `purchase_invoices` es el período fiscal (ej: "01/2026"), distinto de la `fechaEmission` (fecha real del comprobante). El Libro IVA Compras se filtra por período, no por fecha.